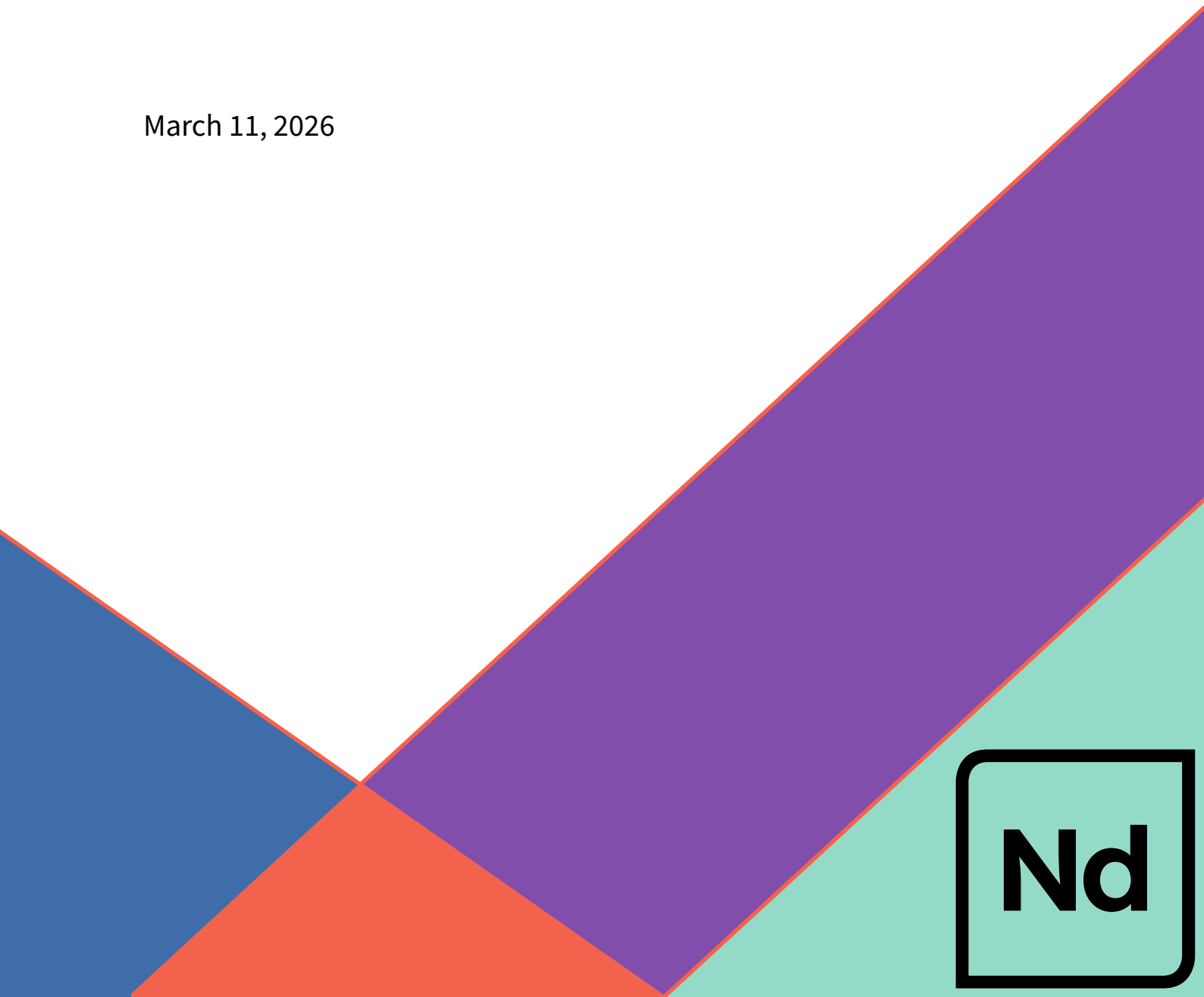

Security Audit - Infinity v2

conducted by Neodyme AG

Lead Auditor:	Jasper Slusallek
Second Auditor:	Nico Gründel
Supporting Auditor:	Benjamin Walny
Administrative Lead:	Jasper Slusallek

March 11, 2026



Nd

Table of Contents

1	Executive Summary	3
2	Introduction	4
	Summary of Findings	4
3	Scope	5
4	Project Overview	6
	General Functionality	6
	Architecture	6
	Changes in Infinity v2	7
5	Findings	11
	[ND-SCT2-IN-01] Limited pausing options	12
	[ND-SCT2-IN-02] Behavior mismatch in which instructions are paused compared to v1 contract	13
	[ND-SCT2-IN-03] Legacy v1 swap instructions use v2 verification	15
	[ND-SCT2-IN-04] Calculation of new withheld lamports may round in unexpected way	17
Appendices		
A	About Neodyme	19
B	Methodology	20
	Select Common Vulnerabilities	20
C	Vulnerability Severity Rating	22

1 | Executive Summary

Neodyme audited **Sanctum's** on-chain Infinity v2 program from January 2026 until March 2026. Neodyme had previously audited the first version of this program, written in another custom framework, **Ideally**. The previous audit report can be found [here](#).

The program was rewritten in the newly created **jiminy** framework. Additionally, several new features and changes were introduced. We defer to the project overview for details on these changes.

The scope of the audit included the entire program. The auditors focused on verifying that behavior matches the old program's implementation including all essential checks, and that the newly introduced changes do not open any new vulnerabilities.

According to Neodyme's [Rating Classification](#), **0 security-relevant** and **4 informational** findings were found. The number of findings identified throughout the audit, grouped by severity, can be seen in [Figure 1](#).



Figure 1: Overview of Findings

The auditors reported all findings to the Sanctum developers, who addressed them promptly. Two were fixed, two were acknowledged and the risk accepted. The security fixes were verified for completeness by Neodyme.

2 | Introduction

From January 2026 until March 2026, Sanctum engaged [Neodyme](#) to do a detailed security analysis of their Infinity v2 project. Two senior security researchers from Neodyme, Nico Gründel and Jasper Slusallek, conducted independent full audits of the contract between the 5th of January 2026 and the 11th of March 2026. Benjamin Walny, an auditor-in-training at Neodyme, did additional auditing work. Both senior auditors have a long track record of finding critical and other vulnerabilities in Solana programs, as well as in Solana's core code itself.

The audit focused on the technical security of the contract, with special emphasis placed on the functionality changes introduced since the version of the program before the major rewrite (whose audit report can be found [here](#)).

Additionally, this report analyzes several economic attack vectors that were present in the old contract and discusses whether they are resolved by the changes.

The rewrite in the new framework is clearly structured. Sometimes the checks are present in locations that are not clear a-priori, and abbreviations in function names are sometimes not intuitively obvious, but this is only a minor hindrance to readability. Overall, although the framework is new, the code can be understood quickly thanks to its coherent layout.

Summary of Findings

All found issues were **quickly remediated**. In total, the audit revealed:

0 critical • **0** high-severity • **0** medium-severity • **0** low-severity • **4** informational

issues.

We would like to emphasize that it is rare that no findings above informational are found. This speaks to careful coding practices and security consciousness on the side of the developers.

3 | Scope

The contract audit's scope comprised of three major components:

- Primarily, the **Implementation** security of the contract's source code, especially verifying that the rewrite did not introduce any new vulnerabilities in previously correct implementation.
- Additionally, security of the **overall design**, with special emphasis on any new attack vectors that may arise due to the functionality changes.
- Relatedly, resilience against **economical attacks**.

Neodyme considers the source code, located at <https://github.com/igneous-labs/inf-1.5/tree/v2>, in scope for this audit. Third-party dependencies are not in scope.

Relevant source code revisions are:

- 5e6cbfac9f6a42795df3dcfedba3c9e145dbe262 · Start of the audit
- 3e2e6bf41a146362eab87c944693b441fc37db79 · Last reviewed revision

4 | Project Overview

This section briefly outlines the changes of Infinity v2 compared to Infinity v1, as well as the functionality and architecture.

General Functionality

Infinity v2, like Infinity v1, is a protocol made for swapping between Liquid Staking Tokens (LSTs).

Infinity is a multi-LST liquidity pool that issues a single LP token (INF). Users can deposit any supported LST and receive INF, the protocol's liquidity token. Later, they can redeem INF for any supported LST. As the LSTs accrue value due to staking rewards in the underlying liquid staking protocols, INF also appreciates in value (when measured in lamport-equivalents).

The pool main functionality of the program is direct LST-to-LST swaps at rates that are determined via the "real lamport value" of the LSTs, as sourced from each of the contracts who mints the respective LST. Additionally, a small fee is charged internally for swaps.

If one LST is near depletion due to high demand, an internal operation can be executed by a privileged rebalance authority which allows lamport-value-preserving external swaps between LSTs to rebalance the LST pools.

Architecture

The general architecture of the program has three types of components: The controller program, the pricing programs and the SOL value calculator programs.

The controller holds the funds managed by the protocol, tracks their value and handles all of the swapping and rebalancing logic. It delegates to the pricing program for fee calculation, and to the SOL value calculator programs for calculating the current lamport value of its LST reserves. The SOL value calculator program infer this value based on external on-chain data stored by the liquid staking protocols responsible for the respective LST.

A quick diagram sketching the relationship of the components is shown below.

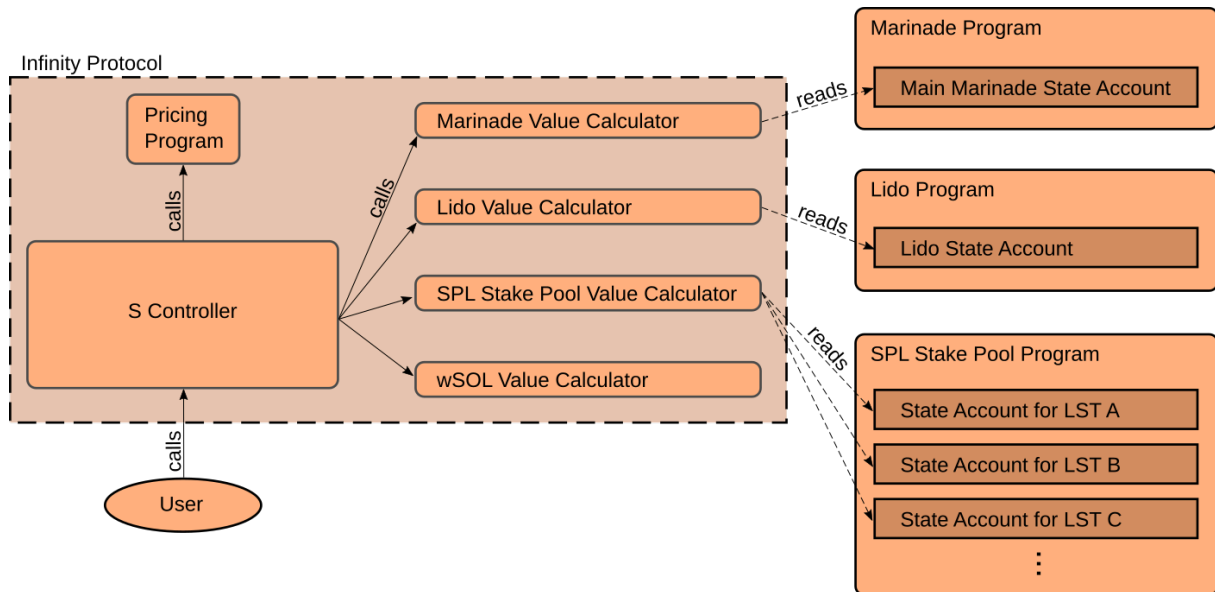


Figure 2: Overview of programs of the Infinity protocol, including their internal and external interactions.

For a more in-depth explanation of the architecture, we refer to Section 2 in our [previous report](#).

Changes in Infinity v2

The major functionality changes introduced in v2 when compared to v1 are as follows:

1. Introduction of **Swap version 2**. Infinity v1 had separate instructions for swapping between supported LSTs and for depositing or withdrawing liquidity in exchange for INF. However, note that the liquidity operations can also be seen as a swap of an LST to INF or vice versa.

In keeping with this, the swap v2 instruction integrates the liquidity operations. They are signaled by setting the LST index to `u32::MAX`, which is hardcoded as the index of INF. Each of the formerly different instructions is handled separately.

2. A **deferred yield system**. In the v1 contract, any change in valuation due to staking rewards is immediately reflected in the internal valuation of INF and therefore in the exchange rates offered for depositing or withdrawing LSTs for INF. This created economic attack vectors that could only be prevented by setting the fees such that any swap covered at least two epochs worth of rewards (see the section on "Synchronization Issues" on page 32 of our old report, as found [here](#)).

The deferred yield system mitigates this problem, along with somewhat limiting the power that external authorities (i.e. authorities in the external liquid staking protocols) have for extracting value from the protocol if they turn malicious. In the deferred yield system, value appreciation from underlying LSTs is only calculated when it is detected and then withheld at first. The withheld value (in lamports) is released and thereby included in the calculation of the value

of INF slowly over time. This is done via an exponential decay of the withheld lamports via a configurable release-per-slot (RPS) rate. A cut of the released lamports goes to the protocol.

The emission of withheld lamports is done via a call to an internal function `release_yield`, which is called before the main logic of all instructions that directly or indirectly

3. **Deferred minting of protocol fees.** In Infinity v1, fees were taken from swaps via deducting them from the output token amount and sending the withheld amount to a token account controlled by the fee beneficiary. There was one such account for each supported LST.

In Infinity v2, this has been changed such that the fee is taken as a percentage of yield releases (i.e. RPS). The yield releases are “funded” by all the types of protocol income (swap, liquidity operations, rewards). The fee beneficiary can mint INF against the fee lamports released to it.

4. **Migration.** Since the new protocol requires new values to be stored on-chain, the `pool_state` account needs to be migrated and the new values added. Specifically, the new `pool_state` account now saves the protocol data version, protocol fee parameters (in a different format), and parameters relating to the RPS mechanism. These are initialized to the obvious values. The migration is idempotent.
5. **Pricing program revamp.** The v1 version only had a flat fee pricing program, which received fee parameters via CPI. Another fee program, the flat slab program, was added, which stores the fee parameters itself in a PDA.

The program was also rewritten in the `jiminy` framework. This framework, which was also authored by the Sanctum developers, tries to optimize for both compute usage and on-chain binary size. A comparison of the framework with anchor and pinocchio can be found [here](#).

Compared to v1, which was written in the `Ideally` framework, the logic of the rewrite is marginally more distributed over the codebase, impacting readability. However, this helps push computation effort to compile-time, thus helping with compute and binary size, as mentioned above.

New Instructions

With the new functionality mentioned above, the following instructions were newly added to the S controller program. For the list of instructions that existed before, please refer to the [previous report](#).

Instruction	Category	Summary
SwapExactInV2	Permissionless	Allows users to swap from any LST to any other LST. They specify the input amount for the swap. Allows the user to specify an Add Liquidity or Remove Liquidity operation by setting either of the LST indices to <code>u32::MAX</code> .
SwapExactOutV2	Permissionless	Same as SwapExactIn, but the user specifies the output amount of the swap, not the input.

Instruction	Category	Summary
SetRPS	RPS authority	To implement the deferred yield system, this allows the RPS authority to set the decay rate of the withheld lamports and thereby control the emission of accumulated rewards.
SetRPSAuth	RPS authority	This allows the RPS to transfer its own authority.
WithdrawProtocolFeesV2	Fee Beneficiary	With the deferred minting of protocol fees being introduced, this instruction lets the protocol fee beneficiary mint the INF that correspond to the lamports that have been accumulated as protocol fees.

Note that the migration functionality does not come with a new instruction. The migration logic is instead executed whenever the `SyncSOLValue`, `swap` or the `liquidity operation` instructions are called. It is idempotent.

Additionally, the `Initialize` instruction, which initializes the globally unique `PoolState` account was removed, since it is no longer needed.

Authority Structure Changes

In addition to the authorities described in the [previous report](#), the **RPS authority** was added.

The RPS authority can change the release-per-slot (RPS) parameter, which controls the emission rate of the deferred yield system. As stated above, the amount of withheld lamports is an exponentially decaying function governed by $\text{new_withheld} = \text{old_withheld} * (1 - \text{pool_state.rps.as_uq0f63}()) ^ \text{slots_elapsed_since_last_update}$ (where `as_uq0f63` interprets an integer as a fraction of 2^{63} – e.g. the integer 2^{62} would be interpreted as 0.5, meaning half of all remaining withheld lamports would be emitted every slot). The RPS authority can change `pool_state.rps` via the `SetRPS` instruction and thereby change the steepness of the exponential decay of withheld rewards.

It can also transfer its own authority to another pubkey via the `SetRPSAuth` instruction.

A compromise of this authority by a malicious actor has very limited effect – it can either set the `rps` value to $\text{MIN_RPS} = 1 / 1_000_000_000_000$, which would mean that emissions practically cease (especially since fees are rounded up). They can also set the RPS to 1, which would enable the limited epoch boundary attacks described in the “Synchronization Issues” section in the [previous report](#).

Economic Attacks

After re-examining all economic attacks described in the “Synchronization Issues” section (see above), we consider all of them solved insofar as they relate to increases in valuation. Previously, users or the rebalance authority were able to front-run value synchronizations to potentially profit from the deltas. By introducing the deferred yield system, it seems likely that this is fully mitigated.

Note that decreases in valuation due to, e.g., slashing could still be front-run.

5 | Findings

This section outlines all of our findings. They are classified into one of five severity levels, detailed in [Appendix C](#). In addition to these findings, Neodyme delivered the Sanctum team a list of nit-picks and additional notes which are not part of this report.

All findings are listed in [Table 2](#) and further described in the following sections.

Identifier	Name	Severity	Status
ND-SCT2-IN-01	Limited pausing options	INFORMATIONAL	Acknowledged
ND-SCT2-IN-02	Behavior mismatch in which instructions are paused compared to v1 contract	INFORMATIONAL	Fixed
ND-SCT2-IN-03	Legacy v1 swap instructions use v2 verification	INFORMATIONAL	Fixed
ND-SCT2-IN-04	Calculation of new withheld lamports may round in unexpected way	INFORMATIONAL	Acknowledged

Table 2: Findings

[ND-SCT2-IN-01] Limited pausing options

Severity	Impact	Affected Component	Status
INFORMATIONAL	Limited pausing options	Pausing functionality	Acknowledged

Currently, there are only two options for pausing:

- Disabling the entire pool, which disables everything save a few select instructions (namely: EnablePool, AddDisablePoolAuthority, RemoveDisablePoolAuthority, SetAdmin and EndRebalance; though to be clear the latter is essentially also disabled because StartRebalance cannot be called)
- Pausing an LST, which disables:
 - all swapping and liquidity operations (including legacy operations) where the contracts would receive that LST
 - Rebalances where the contract would receive that LST

This protects against scenarios where the LST is worth less than what the SOL value calculator program says it is worth. However, it does not protect against the scenario that the LST is worth more. If that is the case, any user can call RemoveLiquidity with that LST and profit.

Whether including RemoveLiquidity in the list of blocked instructions when disabling an LST is of interest depends on what the security model is and whether it is considered likely that an LST will be worth more than reported by its SOL value calculator program.

Resolution

The sanctum developer team acknowledged that this case hadn't been considered. However, they say that this can be mitigated by setting fees in the pricing program high enough to prevent exploits. By setting the fee to 100%, all outflows can be disabled (though this would affect honest users).

[ND-SCT2-IN-02] Behavior mismatch in which instructions are paused compared to v1 contract

Severity	Impact	Affected Component	Status
INFORMATIONAL	Behavior mismatch	Pausing	Fixed

In the v1 of Sanctum S, the instruction SetProtocolFeeBeneficiary requires that the pool not be paused or rebalancing [1]:

```

1
2
3 fn verify_set_protocol_fee_beneficiary<'a, 'info>(
4     accounts: &'a [AccountInfo<'info>],
5 ) -> Result<SetProtocolFeeBeneficiaryAccounts<'a, 'info>, ProgramError> {
6     let actual: SetProtocolFeeBeneficiaryAccounts = load_accounts(accounts)?;
7
8     // [...] <- verifies pubkeys
9
10    let pool_state_bytes = actual.pool_state.try_borrow_data()?;
11    let pool_state = try_pool_state(&pool_state_bytes)?;
12    verify_not_rebalancing_and_not_disabled(pool_state)?; // <- CHECK
13
14    Ok(actual)
15 }
```

[set_protocol_fee_beneficiary.rs, line 25ff](#)

This restriction is not present in v2:

```

1
2
3 pub fn set_protocol_fee_beneficiary_accs_checked<'acc>(
4     abr: &Abr,
5     accs: &[AccountHandle<'acc>],
6 ) -> Result<SetProtocolFeeBeneficiaryIxAccounts<'acc>, ProgramError> {
7     let (accs, _) = accs_split_first_chunk(accs)?;
8     let accs = SetProtocolFeeBeneficiaryIxAccs(*accs);
9
10    let pool = pool_state_v2_checked(abr.get(*accs.pool_state()))?;
11
12    // [...] <- verifies pubkeys
13
14    Ok(accs)
15 }
```

[set_protocol_fee_beneficiary.rs, line 25ff](#)

It is unclear whether this is on purpose, or whether a conscious decision was made to allow changes of the fee beneficiary during rebalancing or pausing.

Setting of the fee beneficiary does not interact in any meaningful way with pausing or rebalancing logic, hence this behavioral mismatch is informational at worst.

Code Locations

- [1] https://github.com/igneous-labs/S/blob/66de438b9e049aacc193b5795d26ac055d86d770/programs/s-controller/src/processor/set_protocol_fee_beneficiary.rs#L43
- [2] https://github.com/igneous-labs/inf-1.5/blob/5e6cbfac9f6a42795df3dcfedba3c9e145dbe262/controller/program/src/instructions/protocol_fee/set_protocol_fee_beneficiary.rs#L24

Resolution

The sanctum developers resolved this finding with [PR 165](#), which has been merged. The fix was verified by Neodyme.

[ND-SCT2-IN-03] Legacy v1 swap instructions use v2 verification

Severity	Impact	Affected Component	Status
INFORMATIONAL	Brittle checks	Swap	Fixed

The contract provides both the legacy `SwapExactIn`, `SwapExactOut`, `AddLiquidity` and `RemoveLiquidity` instructions, as well as the newly added `SwapExactInV2` and `SwapExactOutV2`. The v2 variants also cover adding and removing liquidity by setting one of the LST indexes to `u32::MAX`.

In order to allow this, the `verify_swap_v2` function has a special case that, for LST index `u32::MAX`, allows an arbitrary SOL value calculation program [1]. This is the program that is normally later invoked and the result used in the output calculation. Since no separate SOL value calculation program exists for INF, the validation of that program is skipped.

However, `verify_swap_v2` is also used for `SwapExactIn` and `SwapExactOut` [2]:

```

1 // core user-facing ix
2 // v1 swap + liquidity
3 (&SWAP_EXACT_IN_IX_DISCM, data) => {
4     sol_log("SwapExactIn");
5     let args = parse_swap_ix_args(ix_data_as_arr(data)?);
6     let accs = swap_split_v1_accs_into_v2(abr, accounts, &args)?;
7     let clock = Clock::write_to(&mut clock)?;
8     verify_swap_v2(abr, &accs, &args, clock)?; // <- uses v2 check
9     process_swap_exact_in_v2(abr, cpi, &accs, &args, clock)
10 }
11 (&SWAP_EXACT_OUT_IX_DISCM, data) => {
12     sol_log("SwapExactOut");
13     let args = parse_swap_ix_args(ix_data_as_arr(data)?);
14     let accs = swap_split_v1_accs_into_v2(abr, accounts, &args)?;
15     let clock = Clock::write_to(&mut clock)?;
16     verify_swap_v2(abr, &accs, &args, clock)?; // <- uses v2 check
17     process_swap_exact_out_v2(abr, cpi, &accs, &args, clock)
18 }

```

[lib.rs, line 167ff](#)

This means that one can call either of these instructions with `u32::MAX` as one of the LST indexes, and the sol value calculator program which is then invoked is not checked at all.

Note that this will fail due to array indexing at a later point [3]. However, this is a brittle security architecture for such a fundamental check. There should either be an explicit check disallowing `u32::MAX` in the legacy swap instructions, or a separate verification instruction should exist for these instructions.

Code Locations

- [1] <https://github.com/igneous-labs/inf-1.5/blob/5e6cbfac9f6a42795df3dcfedba3c9e145dbe262/controller/program/src/instructions/swap/v2/common.rs#L175>
- [2] <https://github.com/igneous-labs/inf-1.5/blob/5e6cbfac9f6a42795df3dcfedba3c9e145dbe262/controller/program/src/lib.rs#L174>
- [3] <https://github.com/igneous-labs/inf-1.5/blob/5e6cbfac9f6a42795df3dcfedba3c9e145dbe262/controller/program/src/svc.rs#L43>

Resolution

The sanctum developers stated that they had also noticed this a few days prior. They resolved this by changing the behavior of the v1 swap instructions to now also allow liquidity operations via `u32 : :MAX` in [commit 9614ad3](#). Neodyme verified the fix.

[ND-SCT2-IN-04] Calculation of new withheld lamports may round in unexpected way

Severity	Impact	Affected Component	Status
INFORMATIONAL	Unexpected rounding	Yield release	Acknowledged

In the `ReleaseYield::calc` function, the intermediate value of withheld lamports is calculated via the formula `new_withheld_lamports = (rem_ratio_numerator * withheld_lamports).div_ceil(rem_ratio_denominator)` [1]:

```

1  pub const fn calc(&self) -> YRelLamports {
2      let Self {
3          withheld_lamports,
4          params:
5              ReleaseYieldParams {
6                  rps,
7                  protocol_fee_nanos,
8                  slots_elapsed,
9              },
10     } = self;
11
12     let rem_ratio =
13         rps.as_inner().one_minus().pow(*slots_elapsed).into_ratio();
14
15     // use `Ceil` to round in favour of withholding more yield than necessary
16     //
17     // unwrap-safety: .apply never panics because
18     // - ratio <= 1, so never overflows
19     let new_withheld_lamports =
20         Ceil(rem_ratio).apply(*withheld_lamports).unwrap();
21
22     // [...]
23 }

```

[release.rs, line 89ff](#)

This ceils in order to round in favor of the protocol, i.e. to favor withholding more lamports. However, the calculation of `rem_ratio` is an exponentiation of a `UQ0F63`. Each step of the fast exponentiation algorithm that is executed for this performs a multiplication of a `UQ0F63`, and these multiplications simply round to the nearest integer [2]:

```

1
2
3  pub const fn uq0f63_mul(UQ0F63(a): UQ0F63, UQ0F63(b): UQ0F63) -> UQ0F63 {
4      // == 0.5
5      const ROUNDING_BIAS: u128 = 1 << Q_SUB_1;

```

[uq0f63.rs, line 64ff](#)

```
6
7    // where d = u64::MAX
8    // (n1/d) * (n2/d) = n1*n2/d^2
9    //
10   // as-safety: u128 bitwidth > u64 bitwidth
11   // unchecked arith safety: u64*u64 never overflows u128
12   let res = (a as u128) * (b as u128);
13   // round off 64th bit
14   //
15   // unchecked arith safety: res <= D * D + ROUNDING_BIAS < u128::MAX
16   let res = res + ROUNDING_BIAS; // <- ROUNDING HAPPENS HERE
17   // convert back to UQ0F63 by making denom = d
18   // n1*n2/d^2 = (n1*n2/d) / d
19   // so we need to divide res by d,
20   // and division by 2^Q is just >> Q
21   //
22   // as-safety: truncating conversion is what we want
23   // to achieve floor mul
24   UQ0F63((res >> Q) as u64)
25 }
```

However, the impact is extremely minimal, since the fast exponentiation has at most a few steps.

Relevant Code

[1] <https://github.com/igneous-labs/inf-1.5/blob/5e6cbfac9f6a42795df3dcfedba3c9e145dbe262/controller/core/src/yields/release.rs#L106>

[2] <https://github.com/igneous-labs/inf-1.5/blob/5e6cbfac9f6a42795df3dcfedba3c9e145dbe262/controller/core/src/typedefs/uq0f63.rs#L77>

Resolution

The sanctum team acknowledged the finding, but stated that they would keep it as-is.

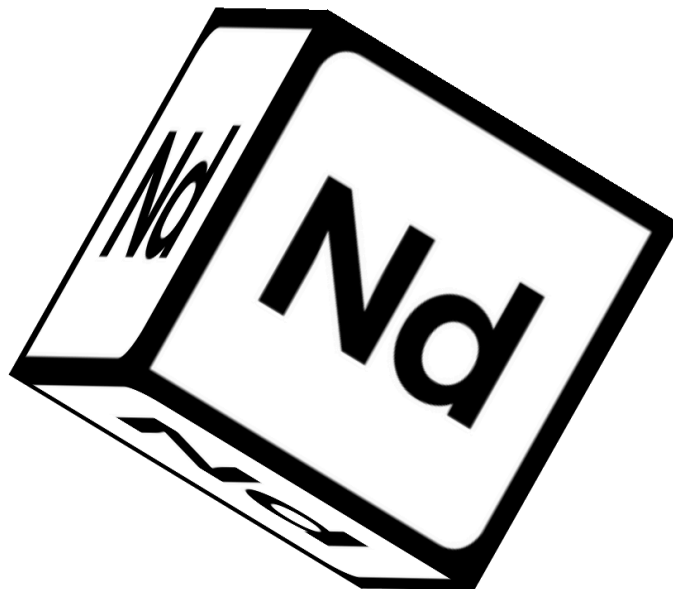
A | About Neodyme

Security is difficult.

To understand and break complex things, you need a certain type of people. People who thrive in complexity, who love to play around with code, and who don't stop exploring until they fully understand every aspect of it. That's us.

Our team never outsources audits. Having found over 80 High or Critical bugs in Solana's core code itself, we believe that Neodyme hosts the most qualified auditors for Solana programs. We've also found and disclosed critical vulnerabilities in many of Solana's top projects and have responsibly disclosed issues that could have resulted in the theft of over \$10B in TVL on the Solana blockchain.

All of our team members have a background in competitive hacking. During such hacking competitions, called CTFs, we competed and collaborated while finding vulnerabilities, breaking encryption, reverse engineering complicated algorithms, and much more. Through the years, many of our team members have won national and international hacking competitions, and keep ranking highly among some of the hardest CTF events worldwide. In 2020, some of our members started experimenting with validators and became active members in the early Solana community. With the prospect of an interesting technical challenge and bug bounties, they quickly encouraged others from our CTF team to look for security issues in Solana. The result was so successful that after reporting several bugs, in 2021, the Solana Foundation contracted us for source code auditing. As a result, Neodyme was born.



B | Methodology

We are not checklist auditors.

In fact, we pride ourselves on that. We adapt our approach to each audit, investing considerable time into understanding the program upfront and exploring its expected behavior, edge cases, invariants, and ways in which the latter could be violated.

We use our uniquely deep knowledge of Solana internals, and our years-long experience in auditing Solana programs to find bugs that others miss. We often extend our audit to cover off-chain components in order to see how users could be tricked or the contract affected by bugs in those components.

Nonetheless, we also have a list of common vulnerability classes, which we always exhaustively look for. We provide a sample of this list here.

Select Common Vulnerabilities

Our most common findings are still specific to Solana itself. Among these are vulnerabilities such as the ones listed below:

- Insufficient validation, such as:
 - Missing ownership checks
 - Missing signer checks
 - Signed invocation of unverified programs
 - Account confusions
 - Missing freeze authority checks
 - Insufficient SPL account verification
 - Dangerous user-controlled bumps
 - Insufficient Anchor account linkage
- Account reinitialization vulnerabilities
- Account creation DoS
- Redeployment with cross-instance confusion
- Missing rent exemption assertion
- Casting truncation
- Arithmetic over- or underflows
- Numerical precision and rounding errors
- Anchor pitfalls, such as accounts not being reloaded
- Non-unique seeds
- Issues arising from CPI recursion
- Log truncation vulnerabilities
- Vulnerabilities specific to integration of Token Extensions, for example unexpected external token hook calls

Apart from such Solana-specific findings, some of the most common vulnerabilities relate to the general logical structure of the contract. Specifically, such findings may be:

- Errors in business logic
- Mismatches between contract logic and project specifications
- General denial-of-service attacks
- Sybil attacks
- Incorrect usage of on-chain randomness
- Contract-specific low-level vulnerabilities, such as incorrect account memory management
- Vulnerability to economic attacks
- Allowing front-running or sandwiching attacks

Miscellaneous other findings are also routinely checked for, among them:

- Unsafe design decisions that might lead to vulnerabilities being introduced in the future
 - Additionally, any findings related to code consistency and cleanliness
- Rug pull mechanisms or hidden backdoors

Often, we also examine the authority structure of a contract, investigating their security as well as the impact on contract operations should they be compromised.

Over the years, we have found hundreds of high and critical severity findings, many of which are highly nontrivial and do not fall into the strict categories above. This is why our approach has always gone way beyond simply checking for common vulnerabilities. We believe that the only way to truly secure a program is a deep and tailored exploration that covers all aspects of a program, from small low-level bugs to complex logical vulnerabilities.

C | Vulnerability Severity Rating

We use the following guideline to classify the severity of vulnerabilities. Note that we assess each vulnerability on an individual basis and may deviate from these guidelines in cases where it is well-founded. In such cases, we always provide an explanation.

Severity	Description
CRITICAL	Vulnerabilities that will likely cause loss of funds. An attacker can trigger them with little or no preparation, or they are expected to happen accidentally. Effects are difficult to undo after they are detected.
HIGH	Bugs that can be used to set up loss of funds in a more limited capacity, or to render the contract unusable.
MEDIUM	Bugs that do not cause direct loss of funds but that may lead to other exploitable mechanisms, or that could be exploited to render the contract partially unusable.
LOW	Bugs that do not have a significant immediate impact and could be fixed easily after detection.
INFORMATIONAL	Bugs or inconsistencies that have little to no security impact, but are still noteworthy.

Additionally, we often provide the client with a list of nit-picks, i.e. findings whose severity lies below Informational. In general, these findings are not part of the report.

Neodyme AG

Dirnismaning 55
Halle 13
85748 Garching
Germany

E-Mail: contact@neodyme.io

<https://neodyme.io>